

PenFlow Coding Standards

1. Introduction

This document establishes our coding standards for the PenFlow project which is a comprehensive platform for managing, scanning as well as analysing security vulnerabilities. These standards ensure consistency, maintainability as well as quality throughout the entire repo.

Project Stack

- Frontend: Next.js + React + TypeScript + Tailwind CSS + shadcn/ui
 - Backend: Python + FastAPI + SQLAlchemy + Celery + Redis + RabbitMQ
 - Database: PostgreSQL
 - Infrastructure: Docker + pnpm
-

2. General Principles

Code Quality

- Write clean, readable and maintainable code.
- Use meaningful naming conventions.
- Keep functions and classes focussed on single responsibilities.
- Use reusable logic, do not repeat code.

Consistency

- Use consistent formatting in each of our three sections (Backend + Workers + Frontend), the format is specific to each section.
- Follow established patterns within the codebase.
- Use automated tooling for code formatting and linting (eslint, ruff, prettier).

Performance

- Optimise for readability then performance.
- Minimise network requests and database queries.
- Implement proper caching and rate-limiting strategies.

Security

- We follow security best practices.
 - Never commit any sensitive information.
 - Validate all user inputs.
 - Use proper authentication and authorization (Keycloak/JWT).
 - Passwords are always hashed.
-

3. Structure and Standards

Backend Standards

```
backend/
|-- app/                # Core python application folder
|   |-- api/            # Entry points for HTTP requests
|   |   |-- middleware/ # Code that runs before requests hit the route
|   |   |-- routes/     # Actual endpoints
|   |   |-- validation/ # Custom Validators
|   |-- config/
|   |-- docs/
|   |-- models/         # SQLAlchemy definitions
|   |-- queue/          # Background processing setup
|   |-- realtime/       # WebSocket handlers
|   |-- repositories/   # Database layer
|   |-- schemas/        # Pydantic models
|   |-- services/       # Business logic
|   |-- templates/
|   |-- utils/          # Shared helpers
|   |-- main.py         # Entry points
|-- scripts/            # Backend specific
|-- tests/              # Pytest testing directory
```

- Naming: snake_case for file naming, functions and variables. PascalCase for classes. Upper snake case for constants.
- Typing: Strict type hints are mandatory.
- Async: Use async/await for all database calls and external network requests
- Architecture: Routes do not contain logic, but rather hands off the work to services/ which interact with the database exclusively through repositories/
- Error Handling: Use centralized logging and custom exceptions. We never return raw errors to the client.

Docker

```
-- postgres/                # PostgreSQL initialization and
testing environment
|   |-- test_backend_integration.sh    # Script to test DB connection from
backend
|   |-- test_workers_integration.sh    # Script to test DB connection from
workers
|-- Dockerfile.backend        # Container blueprint for FastAPI
|-- Dockerfile.frontend      # Container blueprint for Next.js
application
|-- Dockerfile.workers       # Container for blueprint for the
Celery workers
|-- Dockerfile.workers.test
```

Frontend Standards

```

frontend/
|-- public/
|-- src/
|   |-- app/                # Next.js App Router
|   |   |-- api/
|   |   |-- auth/
|   |   |-- dashboard/
|   |   |-- domains/
|   |   |-- health/
|   |   |-- history/
|   |   |-- images/
|   |   |-- login/
|   |   |-- phase2_scan/
|   |   |-- register/      # Account creation
|   |   |-- report/        # Vulnerability report generation and viewing
|   |   |-- scan/          # Scan initiation and realtime status views
|   |   |-- globals.css
|   |   |-- layout.tsx     # Global wrapper
|   |   |-- page.tsx       # Main landing page
|   |-- lib/               # Non UI logic
|   |-- shared/
|   |-- types/
|-- tests/                 # Isolated testing

```

- Naming: Pascal case for React components (Camel case because React uses it natively)
- Components: Use "use client". Default to Server Components.
- Styling: Tailwind CSS. Dynamic classes are managed via cn().
- Architecture: Use functional components with react hooks.

Scripts

```

scripts/
|-- deploy-ecs-service.sh

```

Testing

```

tests/e2e/                # End to end browser testing
|-- specs/                 # Actual test scenarios simulating user behaviour
|-- support/               # Custom commands, global overrides and e2e
configuration

```

- Unit Tests: Test individual functions and utilities in isolation
- Integration Tests: Tests the connection between our different sections (Backend, Frontend, Workers)
- E2E Tests: Test the user workflows in a real browser environment
- Coverage Targets: Maintain high coverage.

Worker Standards

```
workers/
|-- app/                # Core worker application folder
|   |-- monitoring/     # Health checks and metrics for Celery workers
|   |-- queue/          # Task routing and Celery application initialization
|   |-- services/       # Core execution logic
|   |-- tasks/          # What gets triggered by the backend
|   |-- templates/
|   |-- utils/
|-- docs/               # Templates for generating scan results
|-- tests/              # Worker specific testing
```

- Standardised output: Every worker must return data using the strict Phase 2 standard contract.
- Data Normalization: Workers are responsible for translating their specific tool outputs into our unified schema before returning data.
- Workers handle failure gracefully. We wrap tool executions in try/except blocks to update the database status rather than crashing.
- Implement timeouts for all external processes to prevent deadlocks.

Version Control and PR standards

- Branching: All work (whether it be a new feature or documentation) must branch from dev.
- Commits: Although they have to be descriptive, they also have to be to the point.
- Pull Requests: Always ensure you have the latest version of dev in your local branch and all automated checks have to pass locally before making a PR (pnpm test:all, pnpm lint, pnpm build, docker compose up --build)
- PRs to dev require at least two approvals to go to dev and at least 3 to go to main.

Security Standards

- Authentication: JWT, Keycloak
- Authorization: Ensures users can only access their own data. Ownership is always verified before updating or deleting a resource.
- Rate Limiting: Enforced globally on the backend via SlowAPI to prevent DoS attacks on our heavy endpoints.
- CORS: Define allow_origins and wildcard origins are forbidden in production.

API Design Standards

- We use the RESTful principles for endpoints
- Routes are prefixed with /api/v1/...
- Appropriate method usage and proper status codes.
- Use DTOs for all request and response shapes.
- We used Swagger for backend documentation.
- Response shapes should keep consistency, including a success flag, the payload and optional message.

Database Standards

- Table names: Snake case
- Column names: Snake case
- Primary Keys: Use UUIDs for primary keys.
- Relationships: Normalize where appropriate and use foreign keys.